Systematic Literature Review of Semantic Caching for LLM Inference

Kaled Mahmmod Sifat
ID: 2022-3-60-217
2022-3-60-217@std.ewubd.edu
East West University
Dhaka, Bangladesh

Kaniz Reza Mithila
ID: 2022-3-60-052
2022-3-60-052@std.ewubd.edu
East West University
Dhaka, Bangladesh

Redwan Ahmed
ID: 2022-3-60-155
2022-3-60-155@std.ewubd.edu
East West University
Dhaka, Bangladesh

Abstract

Running inference with a large language model can be expensive in terms of both computation and resources. Semantic caching can reduce this cost by reusing previously generated responses for new queries with similar meanings. Research on semantic caching has grown rapidly since 2023, but no published review provides a systematic overview of the field. This paper reviews 66 papers selected from 87 identified records and organizes them into five main clusters. The review identifies five problems: unreliable similarity thresholds, lack of conversation context, incomplete solutions, unmeasured costs of improving reliability, and stale cached information. Across 18 papers reported, speedups range from 1.6 to 3.1 times, while improvements in average latency are generally larger than improvements in tail latency. The review also finds that current studies mainly focus on latency, accuracy, hit rate, and monetary cost, with very limited attention to energy use. Most importantly, none of the reviewed papers measures the energy cost of semantic caching against the energy cost of the LLM inference it replaces. This reveals an important research gap in green computing. Based on this finding, the review identifies three open questions about when semantic caching can provide real energy savings, how cache hit rate affects those savings, and how caching overhead changes energy benefit. The master corpus, quality-appraisal data, and full reference list are available in an accompanying GitHub repository: https://github.com/Sifat-Mahmood/semantic_caching_for_LLM.git

Keywords

Large Language Models (LLMs), Semantic Caching, LLM Inference, Similarity-Based Caching, Adaptive Thresholds, Cache Reliability, Cache Eviction, Energy Efficiency, Green Computing

ACM Reference Format:
Kaled Mahmmod Sifat, ID: 2022-3-60-217, Kaniz Reza Mithila, ID: 2022-3-60-052, and Redwan Ahmed, ID: 2022-3-60-155. 2026. Systematic Literature Review of Semantic Caching for LLM Inference. In *Proceedings of 13th International Conference on Next-generation Computing, Communication, Systems and Security (NSySS 2026).* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

NSySS 2026,

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Large language models (LLMs) are now used for many tasks, such as answering questions, summarizing text, writing code, and helping users with everyday work. As the number of users and queries increases, the cost of running these models also becomes an important concern. Many users may ask questions that are different in wording but very similar in meaning, which can cause the system to perform almost the same computation many times. Semantic caching is one approach that can reduce this repeated work by storing previous responses and reusing them when a new query is sufficiently similar. However, deciding whether two queries are similar enough to share a cached response is not always simple. This review examines how semantic caching has been used for LLM inference, what problems current methods face, and what important questions remain unanswered.

1.1 Research Rationale

Running inference on a large language model is expensive. Every query, even a common one asked a thousand times a day, recomputes the full forward pass from scratch unless something intervenes. Caching is the obvious intervention, and it has worked for databases and web systems for decades. It runs into a specific problem in the LLM setting: two users rarely phrase the same underlying question the same way. "How do I reset my password?" and "I forgot my login, what should I do?" have almost the same meaning, even though they use different words. A cache that only looks for exactly matching text cannot reuse the answer in such cases. Semantic caching solves this problem by comparing the meaning of queries instead of their exact words. Usually, an embedding model changes each query into a vector, and a similarity score is then used to check whether a previous response can be used for the new query. This review focuses on an important benefit of semantic caching: it can help reduce the energy and carbon costs of LLM inference. By reusing previous responses, semantic caching can avoid running the LLM again for similar queries. One paper in our review directly measures the energy and carbon costs of LLM efficiency techniques in industry [16]. It confirms that efficiency claims in this space are not automatically true just because a technique sounds efficient. That paper does not study caching specifically. This review exists partly to ask whether semantic caching's own efficiency claims hold up to the same scrutiny.

1.2 Origins of semantic caching

The term "semantic caching" predates large language models by almost thirty years. Dar et al. [29] introduced it in 1996 for client-server database systems, proposing a caching model based on the

logical content of a query rather than the physical pages or tuples it touched. Chidlovskii and Borghoff [30] extended the same idea to web queries in 2000. Both papers share a property that matters a great deal for this review, they decide whether a cached result can answer a new query using exact logical containment. If a new query's conditions are logically a subset of an already-cached query's conditions, the cache can answer it correctly, provably, every time. There is no approximation and no threshold to tune.

The LLM era kept the name and changed the mechanism entirely. A natural-language query has no formal predicate to check for logical containment. What the field adopted instead is embedding similarity, popularised by GPTCache [20] from 2023 onward. Convert the query to a vector. Compare it against cached vectors. Reuse a cached response if the similarity score clears some threshold. This is a fundamentally different kind of decision from the one Dar et al. and Chidlovskii and Borghoff describe. It is approximate by construction, and no threshold can make it exact. Pattern P1 in this review (Section 4), the finding that no fixed similarity threshold reliably separates a correct cache hit from an incorrect one, traces directly back to this shift. The field imported a name from a paradigm where correctness was guaranteed, into a new paradigm where it is not, and much of the literature this review covers is a twenty-year-later reckoning with that gap.

1.3 Purpose and contribution of this review

Semantic caching for LLM inference has grown quickly since 2023. No published taxonomy currently consolidates this literature into a single, systematic account of what has been tried, what recurs across independent efforts, and what remains genuinely unresolved. This review addresses that gap directly.

We searched three databases, a preprint archive, and a citation index across three rounds, identifying 87 candidate records. Three predate the LLM era or fall outside caching entirely and are used here as background rather than corpus members. Of the remaining 84, full-text screening against five inclusion criteria and five exclusion criteria left 66 papers in the reviewed corpus (Section 2). Those 66 papers organise into five clusters by shared mechanism and shared problem. The corpus core is query-response caching, followed by adaptive thresholds, security, agentic and domain-specific applications, and a structurally distinct KV-cache literature. That last cluster is kept as contrast evidence rather than treated as a sixth variation on the same idea (Section 3). Reading across all five clusters surfaces five patterns that no single paper states on its own (Section 4), developed in full as the clusters are worked through one at a time (Section 5). Eighteen papers report quantitative results precise and comparable enough to place side by side, revealing two patterns visible only in aggregate and one methodological caution about reading headline numbers uncritically (Section 6).

The central finding of this review follows from all of the above rather than from any single paper: across the entire 66-paper corpus, not one reports the energy cost of semantic caching against the energy cost of the direct inference it replaces. For a literature this review approaches from a green-computing motivation, that is not a minor gap. Section 7 develops this into three specific open questions for future work in the field. Section 8 concludes.

2 Methodology

This section explains how the papers for this review were found and selected. It describes the search process, the criteria used to include or exclude papers, and the screening steps used to build the final corpus. It also explains how the quality of each included paper was assessed so that stronger and weaker evidence could be considered appropriately.

2.1 Search Strategy

Literature was identified through five sources: Google Scholar, arXiv, IEEE Xplore, the ACM Digital Library, and Semantic Scholar. Search terms combined the core concepts of semantic caching and LLM inference including variations such as "semantic caching for LLM", "semantic cache", and "green computing LLM inference" which applied iteratively rather than as a single fixed query set. Publication date filtering was not applied consistently across the search process: earlier rounds used a 2024-2026 window, later rounds a broader 2020-2026 window, and citation based checks on already included papers applied no date filter at all. Two papers pre-dating the LLM era were deliberately retained outside any recency filter and are used as historical context in the Introduction rather than as members of the reviewed corpus (Section 3).

Table 1: Search rounds

Round	Purpose	Search approach	n
1	Build the foundational SLR corpus	Pre-filtered, purpose-built search directly targeting semantic caching for LLM inference	30
2	Expand the corpus and Cluster D specifically	Targeted search for agentic, tool-calling, and domain-specific caching literature	29
3	Broaden corpus coverage	Broader keyword sweep rather than a pre-filtered list	28

Total records identified across all three rounds: 87. The search strategy is reconstructed from screening records rather than a preserved query log; exact per-query result counts were not retained at the time of searching. This is stated here directly as a limitation of the search documentation, not concealed.

2.2 Inclusion and Exclusion Criteria

A record was included only if it satisfied all five of the following criteria:

- The paper's core method uses similarity, not exact match, to decide whether to reuse a previously computed LLM output for a new request. That request can be a user query, a tool call, an agent plan, or an in-context example. The goal is always to avoid redoing the same computation. KV-cache and prefix-level papers work differently: they reuse internal state through exact structural correspondence within a single

generation, not similarity across distinct requests. We treat these as architecturally distinct. They appear separately, as contrast evidence in Cluster E, rather than as core inclusions under this criterion (Section 5.5).
- The paper reports actual results. This means a working system with measurements, a proof, an empirical study, or a demonstrated attack. A proposed idea, a survey, or a position paper does not qualify.
- A large language model, or a transformer generative model performing inference, must be the artifact whose computation is being cached. It cannot simply be a downstream consumer of a cache built for something else.
- We read the full text of every paper and confirmed the abstract's relevance ourselves. We did not include any paper on title or abstract judgment alone.
- The paper is the single most complete or most authoritative version of the work in the corpus.

A record was excluded if any one of the following applied:

- The caching operates on a non-LLM artifact, such as images, time-series database rows, storage pages, or a generic vector index, with no LLM inference in the loop.
- The paper shares semantic-caching vocabulary but describes a structurally different mechanism. Examples include cross-model KV-state fusion between two separate LLMs, or caching mentioned only as incidental background rather than as the object of study.
- The record is a duplicate, or a superseded version of a more complete paper already in the corpus.
- The full text was unavailable, unreadable, or did not confirm what the abstract claimed.
- The paper offers no concrete contribution. It is a pure idea, a survey, or a position piece with nothing demonstrated.

We did not screen two pre-LLM foundational papers and one non-caching energy-measurement paper against these criteria. They were never candidates for the reviewed corpus.

2.3 Screening Process

All 87 identified records went into screening. We set three aside as background citations and did not assess them against the inclusion and exclusion criteria (Section 2.2). That left 84 records for full-text screening. We retrieved full text for all 84, a 100% retrieval rate, and assessed each one against the criteria in Section 2.2. Eighteen records were excluded at this stage:

Table 2: Exclusion reasons at full-text screening

Exclusion reason	n
Duplicate of a paper already in the corpus	1
Wrong domain (no LLM inference actually involved)	7
Wrong mechanism (LLM present, but caching applied to the wrong object or a structurally different mechanism)	5
No concrete contribution (survey, position, or demo paper with no evaluated system)	5
Total excluded	**18**

By paper ID, the reasons break down as follows. Duplicate: [31]. Wrong domain: [36], [39], [49], [52], [80], [86], [87]. Wrong mechanism: [75], [76], [77], [81], [85]. No concrete contribution: [73], [78], [79], [82], [83].

That left 66 papers in the review. The three-round search structure explains an asymmetry in these exclusions (Section 2.1). Thirteen of the eighteen total exclusions, or 72%, came from Round 3 alone, even though Round 3 contributed fewer records than Round 1. This is the expected signature of a broader, less pre-filtered keyword sweep. It is also consistent with a rigorously applied set of criteria, not evidence against one. The first two rounds were narrowly targeted, so they yielded a high relevance rate. The third round was deliberately broader, so it surfaced more off-topic material. The same criteria correctly filtered that material out.

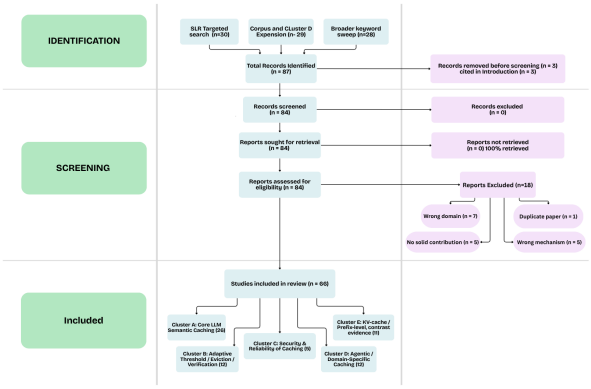

Figure 1: PRISMA-style selection and clustering process

Figure 1 summarises the full flow from identification through to the final included corpus; Table 3 gives the same information as stage counts.

Table 3: PRISMA-style screening flow (stage counts)

Stage	n
Records identified (Round 1 + Round 2 + Round 3)	87
Set aside as background citations (not screened)	3
Sought for full-text retrieval	84
Retrieved (100%)	84
Assessed for eligibility	84
Excluded – duplicate	1
Excluded – wrong domain	7
Excluded – wrong mechanism	5
Excluded – no concrete contribution	5
Included in review	**66**

2.4 Quality-Appraisal Rubric

Because the inclusion and exclusion criteria did not depend on the publication venue or evaluation scale (Section 2.2), all papers were considered using the same criteria. A separate quality assessment was then used for every selected paper. This helped keep lower-confidence sources, such as thesis, minor conference papers, and demo papers, visible in the review. Instead of removing them, they

were given suitable importance based on their quality. Each paper was scored from 1 (low) to 3 (high) using four quality dimensions:

Table 4: Quality-appraisal rubric dimensions

Dimension	1-Low	2-Moderate	3-High
Venue rigor	Unpublished thesis, unrecognized venue, no identifiable venue	arXiv preprint, workshop paper, minor conference	Peer reviewed major conference/journal
Evaluation rigor	Demo, proprietary-only, single toy example	Single dataset, limited baselines	Multiple datasets and/or strong baselines, ablations
Reproducibility	No implementation detail, no code mentioned	Partial detail, no code release stated	Code/data released or thorough enough to reimplement
Limitation transparency	No limitations discussed	Brief or generic mention	Explicit, specific limitations section

We sum the four scores to a total between 4 and 12, then map that total to a tier: Low (4 to 6), Moderate (7 to 9), or High (10 to 12). This tier describes the paper. It is not a further filter for inclusion. A Low-tier paper stays in the corpus, and we report its findings, but we read its claims with correspondingly less weight.

Across all 66 included papers, the distribution came out to 9 High, 47 Moderate, and 10 Low. The ten Low-tier papers match sources we flagged during screening: a thesis-level or unrecognized venue, a demo-level evaluation, or an evaluation restricted to a single proprietary dataset. None were excluded from the corpus on this basis alone. That is consistent with how we separated scope from quality in Section 2.2. We carry tier forward explicitly in the quantitative comparison in Section 6, where we flag individual results by tier instead of presenting them with uniform confidence.

3 Taxonomy

3.1 Classification of the Five Clusters

Every cluster in this review splits on the same underlying question: what is being cached, and by what mechanism. Cluster A is the default case. It caches a full query response using similarity, not exact match. Cluster B does not cache anything new. It refines one specific part of Cluster A's mechanism: how the threshold or eviction decision gets made. Cluster C also does not propose a new caching mechanism. It studies whether an existing one can be broken or defended. Cluster D uses the same core mechanism as Cluster A, but applies it where the object being cached is harder than a chatbot answer: a tool call, an agent plan, a reasoning step, a SQL query. Cluster E is the one real structural break. Its papers reuse internal model state inside a single generation, not a stored response reused across separate requests. We treat that as a different mechanism wearing similar vocabulary, not a fifth variation on the same idea.

3.2 Development of the Taxonomy

We did not start from an existing taxonomy and sort papers into it. No published taxonomy currently consolidates the LLM-specific semantic caching literature, so we built the categories from the corpus itself. We read all 66 included papers, grouped them by shared mechanism and shared problem, and revised the groups until every paper had exactly one clear home. Five clusters is the result of that process, it's not a target we set in advance.

Table 5: Cluster overview

Cluster	Name	n	Role
A	Core LLM Semantic Caching	26	Query-response caching using similarity-based reuse; the corpus core
B	Adaptive Threshold / Eviction / Verification	12	How similarity thresholds and eviction policies are chosen and verified
C	Security & Reliability of Caching	5	Attacks on and defenses of similarity-based caching
D	Agentic / Domain-Specific Caching	12	Tool-calls, agent plans, reasoning steps, and specialised query domains
E	KV-Cache / Prefix-Level Caching	11	Architecturally distinct; retained as contrast evidence, not as core inclusions

Two foundational pre-LLM papers and one non-caching energy-measurement paper sit outside this table entirely. They were never screened as corpus candidates (Section 2.2), and the Introduction cites them only as background.

3.3 Boundary Decisions Between Clusters

3.3.1 ***Cluster C versus Clusters A and B.*** Every Cluster C paper also implements or studies a semantic caching mechanism, the same subject matter as A and B. We separated it anyway, because its contribution answers a different question. A and B papers ask how to make the mechanism accurate. C papers take the mechanism as fixed and ask whether it can be attacked or defended. [42], for example, does not propose a new caching design at all. It demonstrates a key-collision attack against existing semantic caches. That is a different research question about the same object, so it earns its own cluster rather than a subsection of A or B.

3.3.2 ***Cluster D versus Cluster A.*** Cluster D papers use the same similarity-based mechanism as Cluster A. Many of them even reuse an adaptive threshold internally, which is Cluster B's territory. We placed each paper by its core contribution, not by every technique it touches. [19], for instance, extends an existing context-caching approach with a threshold-like mechanism, but its actual claim is about schema-constrained caching for Text-to-SQL systems. Domain adaptation, not threshold design, is what the paper is arguing, so it belongs in D.

3.3.3 ***Cluster E versus the rest.*** This is the boundary we were most careful with, because it is easy to get wrong. Cluster E papers describe their work as "semantic caching," the same term Clusters A through D use. [18] and [28] are typical examples: both compress or recall KV-cache entries at a semantic-cluster granularity, language that sounds close to query-level semantic caching if read out of context. But their unit of reuse is internal state inside one ongoing generation, not a stored response reused across two separate requests. Confusing the two is a real risk in this literature: the vocabulary overlaps completely even though the mechanism does not. We resolved it with a strict test, stated in Section 2.2: does the paper decide, by similarity, whether to reuse a result for a new and distinct request? Cluster E papers fail that test by design, which is exactly why they are retained as contrast evidence instead of folded into the corpus core.

4 Cross-Cutting Synthesis

Reading across all 66 included papers surfaces five patterns that no single paper states on its own. This section names each one plainly and gives its clearest supporting evidence. Section 5 develops the full case for each pattern as it works through the five clusters, paper by paper.

Table 6: Overview of Cross-Cutting Patterns

Pattern	What it means	Clearest evidence	Developed in
P1	The similarity threshold is unreliable	Biton et al. [9]; vCache [22]	Sections 5.2, 5.3
P2	Caching stays blind to context	SmartCache [17]; LLM-Cache [54]	Sections 5.1, 5.4
P3	Each fix improves one thing and leaves the next problem	LLM-Cache [54]	Sections 5.1, 5.2, 5.4
P4	Reliability fixes add unmeasured cost	StepCache [67]	Section 7 (open questions)
P5	Cached results going stale is understudied	CacheSense [26]	Sections 5.1, 5.4

4.1 Pattern 1: Unreliability of Similarity Thresholds

There is no single fixed cutoff that can always tell a correct cache hit from an incorrect one. Biton et al. [9] formally show that finding the best cache-eviction policy for semantic caching is NP-hard. This means that even with complete information, no algorithm can always guarantee the best set of cached responses. vCache [22] shows the same problem from a practical system perspective. The similarity scores of correct and incorrect matches can be very close, so one fixed cutoff cannot reliably prevent wrong matches when the system is used at a large scale. Section 5.2 (Cluster B) explains this problem in detail, while Section 5.3 (Cluster C) shows that it is not only a quality issue but can also create a security risk.

4.2 Pattern 2: Context Blindness in Caching

A similarity decision based only on the current query may miss the meaning of the conversation around it. SmartCache [17] calls this problem "Context Ignorance." The cache makes its decision using only the current query embedding and does not consider the earlier parts of the conversation. LLM-Cache [54] tries to reduce this problem. It improves precision by 8.93% and speed by 2.79× compared with an earlier context-aware method. However, it still reports a "critical limitation" in multi-turn conversations. Sections 5.1 (Cluster A) and 5.4 (Cluster D) show that this problem exists in different settings, from casual chat to more structured domains with strict schemas, and it is still not fully solved.

4.3 Pattern 3: Incremental Improvements and Emerging Limitations

The papers in this review show a similar pattern. Each paper solves one part of the caching problem, but usually does not solve the other related problems. Later papers often find limitations that earlier papers did not address. LLM-Cache [54] is a clear example. It improves performance compared with an earlier method, but it still has a problem with handling multi-turn conversations. Later research can therefore build on this limitation. Sections 5.1, 5.2, and 5.4 show the same pattern in different parts of the caching process, from choosing similarity thresholds to handling agent tool calls.

4.4 Pattern 4: Unmeasured Costs of Reliability Improvements

Making a cache more reliable usually comes with some extra cost, but this cost is often not clearly reported in the main results. StepCache [67] shows this problem clearly. It reports a real improvement in average latency, but its tail-latency results show a different and less visible picture. This issue is discussed in detail in Section 5.2. It also leads to one of the main open questions in Section 7: although several papers report the cost of improving cache reliability, none of them measure this cost in terms of energy consumption.

4.5 Pattern 5: Understudied Cache Staleness

Few papers consider what happens when the information changes after a response has been stored in the cache. CacheSense [26] is a clear example. It uses a freshness-aware cache, but its limitations, discussed in detail in Section 5.1, show that stale information is still an open problem. Section 5.1 (Cluster A) and Section 5.4 (Cluster D) provide further evidence from different domains that this problem has not yet been fully solved.

4.6 Synthesis of the Five Patterns

Three things follow from treating these as a set rather than five separate observations. First, P1, P2, and P5 are corroborated by papers that do not cite or build on one another. That is a stronger form of evidence than repeated citation of one originating claim. Independent research groups, working on different sub-problems,

keep arriving at the same underlying limitations by different routes. Second, the patterns are not independent of each other. Section 5.4 shows P1 and P2 recurring together once caching moves into agentic and domain-specific settings. Third, P4's unmeasured-cost problem is, in the specific case of energy, the direct subject of this review's central open questions in Section 7. No pattern here is presented as solved by any paper in the corpus. Each is presented as evidence of what the field has not yet closed.

5 Thematic Synthesis by Cluster

Section 4 named five patterns that recur across this corpus. The five clusters below are where the evidence for each pattern actually lives. Each cluster synthesis shows what problem its papers share, how the newer additions extend or complicate that problem, and what reading the cluster as a whole reveals that no single paper in it shows alone. Clusters A, B, and D receive full analytical weight. Cluster C's evidence is read alongside Clusters A and B as confirmation from a security angle. Cluster E is presented separately, as contrast evidence rather than as a core inclusion, consistent with its treatment in Section 3.

5.1 Cluster A: Core LLM Semantic Caching

Cluster A is the corpus core. Twenty-six papers share one method: deciding, by embedding similarity rather than exact match, whether to reuse a previously computed LLM response for a new query. This is also where the field's foundational systems live, and three of the five patterns introduced in Section 4 originate here. Everything Clusters B, C, and D go on to specialise or attack traces back to a limitation one of these twenty-six papers admitted first.

5.1.1 ***Reference Implementations and the Baseline Problem.*** GPTCache [20] is the field's reference open-source implementation. Its own future work says plainly that more investigation is needed to balance cache hits against false positives, and its best-case hit rates stay under 90%. That admission is the seed of Pattern P1. GPT Semantic Cache [21] makes the same problem concrete. It uses a fixed similarity threshold of 0.8, and the authors say directly that this threshold "may not generalize." They name dynamic threshold adjustment as unsolved future work. Context-based Semantic Caching for LLM Applications [14] belongs beside these two, but for a different reason than its early date suggests. It is a short paper with no formal limitations section, and past reviews of this corpus undersold it as a result. Read closely, it reports something concrete: a context-aware design that resolves and persists end-user context alongside the query embedding, rather than caching only context-free queries, and the authors report this reduces average response time by over 80% without hurting response quality. That is a real, citable result sitting inside a paper that looks, on the surface, like a design sketch.

5.1.2 ***Context Blindness: Evidence for Pattern P2.*** A second group of papers shows that using only the current query is not enough to understand the full meaning of a conversation. ContextCache [4] uses a two-step retrieval process. First, it matches the query vector. Then, it uses self-attention to look at the previous conversation and improve accuracy in multi-turn dialogues. However, the authors report that earlier context-based methods can suffer from "attention dilution" and "representation flattening." This means that adding more context does not always improve the result, so the problem is only partly solved. SmartCache [17] describes the same problem more directly. It identifies two issues: "Context Ignorance," where the cache looks only at the current query embedding, and "Decoupling from Inference State," where the cache does not fully consider the state of the previous interaction. LLM-Cache [54] improves on ContextCache with clear results. It achieves 8.93% higher precision and a 2.79× speedup. However, it still reports a "critical limitation" for multi-turn conversations. The same query can need different answers depending on the previous conversation, which the basic design cannot fully handle. This clearly shows Pattern P3: one paper improves one problem, while another problem remains for later research. SISO [38] takes this problem further. It argues that the common assumption that similar inputs will produce similar outputs does not always work for multi-turn and context-dependent queries. MeanCache [13] addresses the problem from a deployment perspective. It uses federated, on-device caching and checks the conversation context to reduce false cache hits. However, this approach also has costs, such as running embedding models on the device and dealing with limited storage on client devices. Knowledge Graph-Enhanced Semantic Cache [27] uses a different approach. It adds a knowledge graph to similarity matching and reports up to 28% improvement in accuracy. However, the authors also point out that the knowledge graph mainly captures clear and direct relationships. It may still fail when a query requires more complex or indirect reasoning. Therefore, even this targeted solution does not fully solve the gap described by Pattern P2.

5.1.3 ***Threshold and Embedding Reliability: Evidence for Pattern P1.*** Another group of papers looks at reliability from a different angle. Instead of focusing on conversation context, these papers ask whether the similarity measure itself can correctly identify when two queries have the same meaning.Semantic Caching of Contextual Summaries [23] points out the limitations of GPTCache. Its proposed solution stores intermediate summaries instead of complete responses. However, the authors admit that this approach is still domain-specific and needs careful tuning. Advancing Semantic Caching with Domain-Specific Embeddings [24], using Redis and Virginia Tech's LangCache-Embed, looks at the problem from another side. General-purpose embedding models may not understand some domain-specific queries correctly. For example, "myocardial infarction treatment" and "how to treat a heart attack" have the same meaning, but a general embedding model may not always recognize them as equivalent. This shows that the problem is not only about choosing the right similarity threshold. The embedding model must also represent the true meaning of the queries. LLMs for Test Input Generation [25] uses testing to find these problems. Its VaryGen system uses LLMs to create difficult query pairs that can cause false cache hits or missed matches in GPTCache. The authors conclude that adapting the system for different domains is still a manual and unsolved task. InstCache [33] takes a prediction-based approach. It reorganizes the instruction representation space to improve cache reuse and reports a 2.3× higher hit rate than the upper bound of a traditional cache. However, the system needs a large amount of data. Its performance improves as more traffic is

collected, creating a cold-start problem for new systems. Two other papers show similar problems in practical settings. Non-Standard Query Caching middleware [58] increases the hit rate from 11% to 58% by first normalizing informal queries before matching them. Real-Time Query Understanding [61] reports several limitations. About 11.2% of misses are caused by poor embedding quality for domain-specific terms and new entities. A fixed 0.92 threshold also causes a 2.7% false-positive rate. In addition, the system needs 72 hours for cold start and about 147 GB of RAM for every 100 million embeddings. The Brazilian EdTech evaluation [62] provides another real-world result. Using a fixed 0.98 similarity threshold reduces response time by 3.93× to 12× in a production distributed system. However, instead of implementing an adaptive threshold, the authors leave it as future work. This again shows that choosing the right similarity threshold remains an open problem.

5.1.4 ***Freshness and Staleness: Evidence for Pattern P5.*** Two papers in Cluster A show Pattern P5, which is later extended by papers in Cluster D on OLAP and recommendation systems (Section 5.4). CacheSense [26] is the first paper in this review to clearly show this pattern. It uses a freshness-aware cache that removes stored responses when the original documents change. However, it still has some limitations. It depends on document-level change signals, tests only one threshold at a time, and does not yet support multi-turn conversations. These limitations show that stale cached information is still an open problem. Risk-Constrained Freshness-Aware Semantic Caching [45] studies the same issue in open-web retrieval-augmented generation (RAG). It achieves 95.5% overall answer accuracy, but the authors note that the training data does not fully represent how real-world facts change over time. The system also has a p95 fetch latency 3.5× higher than the median, showing that some requests can take much longer than normal. A similar tail-latency problem also appears in Cluster D as Pattern P4 (Section 4)

5.1.5 ***Systems and Efficiency Innovations.*** The remaining papers extend Cluster A along systems and efficiency lines rather than the reliability or freshness threads directly. Semantic Caching for Low-Cost LLM Serving [12] frames cache eviction as a combinatorial-bandit problem, unifying offline learning and online adaptation. It only partly relaxes the common assumption that query arrival and cost distributions are known in advance. A Generative Caching System [34] (Cisco) goes further than reuse: it synthesises new responses from multiple cached entries to answer queries it has never seen before, and runs faster than GPTCache. It is constrained by its vector-database backend's field-size limits for non-text data. LLM Reproducibility using Three-Way Caching [47] combines exact matching, semantic matching, and a secondary-LLM causal-verification step to improve response consistency. The paper never quantifies the verification step's own latency and cost against the caching benefit it is meant to protect. SemCache [50] moves the problem to the edge. It shares semantic caches across multiple users of LoRA-adapted models, reporting up to 76.7% hit rate and a 1.31x speedup, while noting that multi-user cache coordination at the edge remains generally unsolved. IC-Cache [55] redefines the unit of caching itself. It reuses in-context few-shot examples rather than query-response pairs, cutting serving latency by 28 to 71%, though its relevance-based example selection correlates only weakly with true example helpfulness. Cortex [6] caches semantic knowledge across regions for LLM agents, using approximate nearest-neighbour search plus an LLM-judge verification step. It leaves "more advanced dynamic partitioning" as future work, and does not fully characterise the cost the judge step adds. SCALM [15] is the cluster's most direct challenge to how the field measures success at all. Built on the first large-scale analysis of real human-to-LLM interaction logs, it states outright that "the commonly used cache hit ratio metric is not always effective in calculating cost savings." That caution applies to every hit-rate figure reported elsewhere in this cluster. An undergraduate thesis evaluating semantic caching in production LLM-powered features [46] closes the cluster with an applied data point, a 4 to 10x latency improvement, rather than a new mechanism. It is included per the quality-appraisal rubric (Section 2.4) with correspondingly reduced weight.

5.1.6 ***Cluster Synthesis.*** Read as a whole, Cluster A's twenty-six papers do not converge on one unresolved question the way Cluster B converges on threshold reliability. Instead, they originate three of the five cross-cutting patterns synthesised in Section 4. The reference implementations [20], [21] and their direct critiques [23], [24], [25] establish Pattern P1. The context-aware fixes [4], [17], [54], [38], [13], [27] establish Pattern P2, and LLM-Cache's explicit improvement-yet-still-limited result [54] gives the clearest single illustration of Pattern P3 in the corpus. CacheSense and the open-web RAG paper [26], [45] establish Pattern P5. The systems-and-efficiency papers [12], [34], [47], [50], [55], [6], [15], [46] extend the cluster's reach without resolving any of these three patterns outright, and SCALM's [15] objection to the hit-rate metric itself suggests that some of the improvements reported elsewhere in this cluster may be less comparable than their headline numbers imply.

5.2 Cluster B: Adaptive Threshold and Eviction Policy

Cluster B comprises twelve papers and remains the strongest evidence in the corpus for Pattern P1 (Section 4): no fixed similarity threshold reliably separates a correct cache hit from an incorrect one.

5.2.1 ***The Central Threshold-Reliability Case.*** vCache [22] remains the central paper in this group. Rather than a single fixed cutoff, it computes a statistically justified cutoff per query. Its authors prove that similarity scores for correct and incorrect cache matches overlap substantially, and conclude that no single fixed cutoff below the maximum possible value can reliably avoid mistakes at scale. Biton et al. [9] gives the sharpest theoretical grounding for why this is unavoidable rather than merely difficult in practice. The paper proves that finding the mathematically optimal offline cache-eviction policy for semantic caching is NP-hard. This result matters beyond eviction policy specifically. It establishes that even with perfect hindsight about which future queries will arrive, no efficient algorithm is guaranteed to find the optimal set of cached responses to keep. Every adaptive-threshold and eviction-policy paper in this cluster is, in effect, proposing a tractable approximation to a problem that is provably intractable in its exact form, and Biton et al. is the paper that makes this explicit rather than implicit.

5.2.2 ***Static-Tier and Category-Level Approaches.*** Krites [10] (Apple) describes a "similarity grey zone," a region near the cutoff where the score genuinely cannot distinguish two queries that mean the same thing from two that merely use similar wording. Its fix operates only on previously stored entries, checked later in the background by a separate model. The live, real-time decision at query time is left unchanged and still uses the same unreliable method. Category-Aware Semantic Caching [11] (IBM) instead varies the cutoff by query category, which raises overall hit rate but comes with two costs the authors quantify directly. Roughly 40% of traffic is judged not worth caching under their own cost-benefit analysis, and most production vector databases only expose similarity scores after the search has completed, so a category-varying cutoff requires either repeated searches or multiple parallel databases.

5.2.3 ***Theoretical Limits of Threshold Reliability.*** Two theory-oriented papers extend this picture. Beyond Biton et al.'s eviction-policy result, a companion paper [8] builds a mathematical framework for the case where the space of possible queries is not fixed and finite but effectively unbounded, an endless range of ways to phrase the same intent. Neither paper attempts to make the threshold better for energy efficiency or deployment simplicity. Both treat the threshold as a fixed input to a separate, purely mathematical problem, reinforcing that the reliability question in this cluster has a hard theoretical floor, not just an engineering one.

5.2.4 ***Representation, Calibration, and Fine-Grained Reuse.*** The newer additions push the cluster in two further directions. An Ensemble Embedding Approach [37] fuses several low-correlated embedding models through a trained meta-encoder to improve hit and miss classification accuracy, trading deployment simplicity for improved representation quality. This addresses the representation side of the same reliability problem rather than the threshold side. Closing the Calibration Gap [70], built on production caching infrastructure (LangCache and Redis), targets threshold calibration precision and recall directly against a large sentence-pair benchmark, and is one of the few papers in the cluster to report calibration metrics in a form directly comparable across systems (Section 6). Reuse, formally Continuous Vector Similarity Search, [60] reframes the reliability question for multi-turn and multi-hop query streams specifically. It formalises when a Reuse Trigger should fire at all, a decision distinct from, but closely coupled to, the similarity-threshold problem the rest of the cluster addresses for single-turn queries. StepCache [67] and MVR-cache [69] push the unit of reuse below the whole-response level. StepCache reuses individual reasoning steps with a lightweight verification pass. MVR-cache learns to segment prompts into semantic units for finer-grained matching. Both implicitly argue that the reliability problem is partly an artifact of treating an entire query as one indivisible unit for similarity comparison.

5.2.5 ***Limitations of Recent Approaches.*** LACACHE [53] and the collision-aware semantic hashing proposal [51] close out the cluster, and each is worth reading past its headline result. LACACHE offers a statistical-guarantee approach that reduces per-query latency overhead by 1.6x against a comparable guarantee-providing baseline, but its own authors name three specific gaps left open: the formal guarantee does not cover subtle misinformation attacks that preserve semantic proximity to the genuine response, the theoretical detection-amplification bound may not fully hold when embedding models are architecturally correlated rather than diverse, and the method addresses integrity attacks only, leaving privacy guarantees against timing side-channels as a separate open problem. The collision-aware hashing proposal offers an alternative to standard LSH bucketing meant to ensure semantically similar items are not missed by the hashing scheme itself, but it includes no dedicated limitations section and no discussion of its own weaknesses. Formatting artifacts in its source PDF suggest a lower-maturity, likely non-peer-reviewed write-up, consistent with its Low quality tier under this review's rubric (Section 2.4).

5.2.6 ***Cluster Synthesis.*** Read together, the cluster's twelve papers converge on the same conclusion from three different directions: mathematics (Biton et al., the unbounded-query-space paper), systems engineering (vCache, Krites, Category-Aware Semantic Caching), and finer-grained matching design (StepCache, MVR-cache, the Ensemble Embedding Approach). There is no single fix, at any level of the stack, that resolves the threshold-reliability problem outright. Each paper narrows one facet of it while leaving the rest open.

5.3 Cluster C: Security and Reliability of Caching

Cluster C is the smallest cluster in the review, with only five papers. These papers do not introduce a new caching method. Instead, they show that the problem found in Clusters A and B is more than just an accuracy problem. It can also create a security risk that attackers can use. They all point to the same main problem: a fixed or almost fixed similarity threshold cannot always distinguish between a genuinely similar query and a carefully designed malicious query. This is the same basic problem shown by Biton et al. [9] and vCache [22] in Cluster B.

5.3.1 ***Attacks on Semantic Caching.*** Cache Me, Catch You [3] is the cluster's broadest contribution. It is the first systematic study of cache-related attack vectors, spanning both prefix-level and semantic caching, across four production-representative serving frameworks: vLLM, SGLang, GPTCache, and AIBrix. Its most consequential finding is not a specific exploit. It confirms that semantic caching is already in production use at Google, Azure, AWS, and Alibaba, which means the vulnerabilities this cluster documents are not theoretical. When Cache Poisoning Meets LLM Systems [32] narrows to a specific attack class, semantic cache poisoning, again evaluated across major cloud providers. Existing adversarial-prompt defenses perform poorly, with F1 scores as low as 0.50 to 0.53. The authors propose an improved defense that reaches an F1 of 0.87, but even that does not fully mitigate the attack. From Similarity to Vulnerability [42] gives the most direct confirmation of the threshold reliability problem's security consequences. Its key collision attack achieves up to a 90.6% hit rate for malicious cached queries across multiple backend LLMs. The paper frames this explicitly as an empirical, adversarial confirmation of the same similarity-overlap weakness vCache [22] proves mathematically in Cluster B.

5.3.2 ***Defensive Approaches.*** The remaining two papers focus on defense methods, but both provide only partial solutions. Enhancing Adversarial Resilience in Semantic Caching [7] focuses on protecting retrieval-augmented generation caches. It uses a cluster-centroid method to detect these attacks. However, the authors point out that using only vector similarity is not enough because it mainly looks at the distance between embeddings and may not fully understand the real meaning of a query. The paper also does not measure the extra computing cost of its defense method. GuardCache [57] is designed for FinTech systems. It improves the average F1-score from 0.86 to 0.91 and improves jailbreak detection by 12 percentage points. However, it does not improve response time much for some risky queries. The authors also say that the system needs to understand more of the query context. This is similar to Pattern P2 in Clusters A and D, but here the lack of context creates a security problem, not just an accuracy problem.

5.3.3 ***Cluster Synthesis.*** No paper in Cluster C claims to have solved semantic cache security. The strongest defensive result [57] still leaves specific attack categories only partially mitigated, and the strongest attack result [42] succeeds in the overwhelming majority of attempts against production-representative systems. Read together with Cluster B, this cluster's central contribution to the review is evidentiary rather than architectural. It shows that Pattern P1's threshold-reliability problem is not a purely internal quality concern that only affects hit-rate metrics. It is a property of the caching mechanism itself that an adversary can deliberately locate and exploit, and real production systems [3] are already exposed to it.

5.4 Cluster D: Agentic and Domain-Specific Caching

Cluster D supports a substantive discussion of agentic and multi-turn tool-caching, drawing on twelve papers that apply the corpus's core mechanism to harder objects than a chatbot answer: a tool call, a reasoning step, an execution plan, a specialised query domain.

5.4.1 ***Agents and Stricter Domains.*** ToolCacheAgent [1] studies caching for AI agents invoking external tools, a calculator, a search function, rather than caching a plain conversational answer. SchemaAware-Cache [19] applies semantic caching to SQL query generation, a domain with materially stricter correctness requirements than casual conversation. SchemaAware-Cache's finding matters beyond its own domain: ContextCache's [4] context-combining approach still shows "attention dilution" and "semantic flattening" even in this stricter setting. That suggests the context-blindness problem identified in Cluster A was never actually solved. It was only harder to notice in looser domains.

5.4.2 ***Tool Calls, Reasoning Steps, and Agent Training.*** ToolCaching [40] is the direct successor to ToolCacheAgent's problem framing, and the first dedicated framework for caching LLM tool-calling results specifically. It reduces end-to-end latency by up to 34% for informational tool calls, and shares ToolCacheAgent's core limitation: argument-based invalidation cannot capture hidden or indirect state dependencies between tools. TVCache [66] addresses a related but distinct problem in agent training rather than agent deployment. It caches external tool-call results during reinforcement-learning post-training, so GPUs are not left idle during multi-second or multi-minute tool calls. Its hit rates vary widely by workload, from 14.2% to 73.9%. SemanticALLI [41] moves the unit of caching further still, from tool-call results to intermediate reasoning steps within an agent's own reasoning trace. It reports an 83.10% hit rate in its synthesis stage, though its evaluation is restricted to a single proprietary production dataset, a limitation reflected in its quality-appraisal score (Section 2.4).

5.4.3 ***Whole Execution Plans.*** Agentic Plan Caching [48] generalises one level higher again, caching and adapting entire agent execution-plan templates across semantically similar tasks rather than individual tool calls or reasoning steps. It is the strongest-evaluated paper in this sub-area: a 50.31% cost reduction and 27.28% latency reduction while retaining 96.61% of optimal task accuracy. Its own stated limitation, that benefits diminish for highly dynamic workloads with frequent task variation, is echoed independently by Temporal Semantic Caching in Agentic Plan-Execute Pipelines [68]. That paper targets parameter-rich industrial agent queries with a reranker-judge verification stage, and states plainly, in its own limitations section, that pure semantic similarity is not a sound proxy for answer validity in parameter-rich queries, and that no similarity threshold can fully resolve this. That is an independent, domain-specific confirmation of vCache's [22] core mathematical claim in Cluster B, arrived at from an agentic-systems angle rather than a theoretical one.

5.4.4 ***Specialized Query Domains.*** The remaining additions extend the cluster's domain-specific arm. Semantic Caching for OLAP via LLM-Based Query Canonicalization [43] directly extends SchemaAware-Cache's database angle. It reports an 82% hit rate against 28% (plain text match) and 56% (AST match) baselines, and an 85 to 90% reduction in backend compute. Its own limitations section is unusually candid: window functions and CTEs bypass the cache entirely, natural-language canonicalization accuracy drops under ambiguity, and freshness and update-frequency modeling is explicitly left as future work. That last point connects this paper directly to Pattern P5 (Section 4) as well as to Cluster B's threshold-reliability thread. ChronosBI [59] extends the same database-adjacent reasoning to LLM-powered business-intelligence pipelines. SCaLRec [64] applies the cluster's caching logic to a genuinely different domain again, cloud-device sequential recommendation, where it identifies and corrects "cloud semantic staleness" in cached user embeddings. That gives a third, independent source for the staleness pattern discussed in Section 4. Demo of SemWeave [63] and Adaptive Web Resource Parsing [56] round out the cluster. SemWeave uses an NLI-containment-based reuse mechanism for exploratory data-analytics pipelines, conceptually closer to the field's pre-embedding logical-containment origins than to threshold-based similarity. Adaptive Web Resource Parsing applies LLM-based semantic caching to web-page parsing and structuring.

5.4.5 ***Cluster Synthesis.*** Taken as a whole, Cluster D shows that the field's central problem, reliably deciding when a cached result may be reused, reappears at every level of agentic systems. It shows up at the level of a single tool call (ToolCaching [40]), a single reasoning step (SemanticALLI [41]), a whole execution plan (Agentic Plan Caching [48], Temporal Semantic Caching [68]), and

a specialised query domain (the OLAP [43] and BI [59] papers). No paper in this cluster claims to have solved the problem in general. Each narrows it to a specific agentic or domain context.

5.5 Cluster E: KV-Cache and Prefix-Level Caching

Cluster E consists of eleven papers that this review deliberately does not treat as core corpus members. These papers reuse internal model state through exact structural correspondence within a single generation, the same KV-cache entries for the same ongoing sequence, rather than deciding, by similarity, whether to reuse a response across two distinct requests. Cluster E exists in this review to make the boundary explicit, not to receive the same analytical weight as Clusters A through D.

5.5.1 ***Single-Request and Cross-Query Distinction.*** Two pairs of papers make the distinction easiest to see, because they solve closely related problems on opposite sides of it. ChunkKV [18] and ClusterKV [28] both compress or recall KV-cache entries at semantic-cluster granularity rather than fixed-size pages, language that, read in isolation, sounds close to "semantic caching." But both solve single-request memory footprint and recall efficiency for one ongoing generation, not reuse across different queries. ClusterKV is included specifically as a second example that "semantic" KV-cache work is a consistently separate research thread from query-level semantic caching. This reinforces why small modifications to KV-cache mechanisms would not resolve the reliability. SentenceKV [35] extends the same family to sentence-level granularity, reporting 97.5% accuracy while cutting memory overhead. Its own stated distinction is explicit: it manages internal memory for one answer, not reuse across different questions.

5.5.2 ***Compression and Eviction Mechanisms.*** The remaining papers improve KV-cache compression in different ways. However, these methods focus on making the KV cache smaller or faster. They do not focus on reusing cached responses across different queries. Not All Tokens Are Worth Caching [44] (SAECache) uses semantic information to decide which tokens should be removed instead of using fixed rules. However, it has a cost. On workloads with mostly single-turn queries, it increases the time to get the first token by 12% to 34% because managing multiple queues can take more time than the method saves. DynSplit-KV [65] uses dynamic splitting instead of fixed semantic boundaries. It achieves up to a 2.2× speedup and has less accuracy loss than the fixed method. However, its dynamic weighting also adds extra processing time, which is not fully measured. LLMCache (layer-wise) [71] reuses intermediate model results for similar inputs, not only identical ones. It can work at different transformer layers and reports a 3.1× speedup with less than 0.5% accuracy loss. However, the cache hit rate becomes lower when the input is very different from the training data or when inputs vary a lot. Entropy-Guided KV Caching [72] keeps tokens that receive high attention during the prefill stage. This also helps show which tokens are important. However, some tokens may become important later during decoding, and the method may miss them. It also uses the same top-k limit for all attention heads, which may not work well because different heads can have different needs. HeteroCache [74] gives different cache sizes to different attention heads based on how stable and repetitive their information is. It achieves a 3× decoding speedup. However, the current system does not reach its full theoretical speed because it does not use custom CUDA kernels. Its performance can also be limited by the connection speed between the CPU and GPU, especially when bandwidth is low.

5.5.3 ***Boundary-Adjacent Papers.*** Three further papers sit closest to the Cluster A and E boundary without crossing it, each explicitly distinguishing its own contribution from query-level semantic caching. Cache Your Prompt When It's Green [2] sizes KV and prefix caches for carbon awareness, balancing operational against embodied (SSD) carbon costs. Its carbon model is built specifically around SSD embodied carbon rather than the embedding-and-approximate-nearest-neighbour overhead that defines semantic caching, which makes it a useful contrast paper for cost modeling. Rethinking I/O Caching for Mobile Platforms [5] caches model weight files at the operating-system I/O buffer level, and distinguishes itself directly in its own text by noting it "cannot ensure exact I/O-level reuse," the opposite direction of imprecision from the similarity-based approximation Clusters A, B, and D all study. Accelerating Local LLMs on Edge Devices [84] shares intermediate processing states across edge clients, using partial prompt-similarity matching plus a Bloom-filter catalog to reduce unnecessary network communication. This is structurally closer to semantic caching than the rest of Cluster E, but its unit of reuse is still a shared processing state across clients working on related prompts, not a stored response to a completed, independent query. Its own limitation is that communication overhead is only partially offset by the Bloom filter, with effectiveness still depending on genuine similarity existing between clients' prompts.

5.5.4 ***Cluster Synthesis.*** No paper in Cluster E claims to perform semantic caching in the sense this review defines it, and none should be read as evidence for or against Patterns P1 through P5. Their collective value to this review is definitional. Eleven independent research groups, working on measurably different problems (carbon accounting [2], mobile I/O [5], token eviction [18], [28], [44], cross-layer compression [65], [71], [72], [74], sentence-level memory [35], and edge state-sharing [84]), all describe their contribution using the same "semantic" vocabulary that Clusters A, B, and D use for cross-query reuse, while each explicitly or structurally distinguishes their own single-request mechanism from it. This consistent, independently-arrived-at boundary is itself evidence that the two are genuinely separable, not a reflection of ambiguity in this area of research.

6 Meta-Analysis

A strict statistical meta-analysis is not suitable for this review because the 66 papers use different models, datasets, and workloads. They also do not use one common measure that can be compared across all studies.Instead, this section uses a common approach for computer science literature reviews. Papers that report clear and comparable results, such as hit rate, latency, speedup, accuracy, or precision, are listed together when they compare their results with a stated baseline. This helps us see common patterns across different systems without saying that all results are directly comparable.

Table 7: Quantitative results across selected papers

Paper	System	Metric	Result	Baseline	Tier
[11]	Category-Aware Semantic Caching	Traffic viability	40% of traffic not worth caching	category cost/benefit analysis	High
[33]	InstCache	Hit rate / latency	2.3x hit rate; 42 to 50% time-per-token reduction	traditional instruction cache upper bound	High
[38]	SISO	Hit ratio	1.71x higher hit ratio	vLLM (state of the art)	High
[43]	Semantic Caching for OLAP	Hit rate / compute	82% vs. 28% / 56%; 85 to 90% compute reduction	text-match, AST-match	High
[44]	SAECache	Latency (TTFT)	12 to 34% regression on single-turn-dominated workloads	fixed heuristic eviction	Moderate
[45]	Risk-Constrained Freshness-Aware Caching	Accuracy / latency	95.5% accuracy; p95 2771ms vs. median 915ms	open-web RAG workload	Moderate
[48]	Agentic Plan Caching	Cost / latency / accuracy	50.31% lower cost, 27.28% lower latency, 96.61% accuracy retained	no-cache agent baseline	High
[54]	LLM-Cache	Precision / speedup	8.93% higher precision, 2.79x speedup	ContextCache	Moderate
[58]	Non-Standard Query Caching	Hit rate / latency	11% to 58% hit rate; 4494ms to 17.98ms	cloud baseline, threshold 0.65	Low
[60]	Reuse (Continuous Vector Similarity Search)	Throughput	1.6 to 3.0x higher throughput	standard semantic caching	Moderate
[61]	Real-Time Query Understanding	Hit rate / accuracy	12.4% to 67.8% hit rate; 94.3% accuracy	fixed threshold 0.92	Moderate
[62]	Brazilian EdTech evaluation	Response time	3.93 to 12x reduction	non-caching baseline, threshold 0.98	Moderate
[65]	DynSplit-KV	Speedup / accuracy	Up to 2.2x speedup with less accuracy loss	rigid KV-cache splitting	Moderate
[66]	TVCache	Hit rate	14.2% to 73.9% across workloads	RL post-training task	Moderate
[67]	StepCache	Latency	Mean 2.13s to 0.67s; p95 3.38s to 3.30s	no step-level reuse	Moderate
[68]	Temporal Semantic Caching	Speedup / latency	1.67x speedup, about 40% median latency reduction	no-cache agent pipeline	Moderate
[70]	Closing the Calibration Gap	Precision / recall	Threshold-calibration study, K=50 candidate pool	LangCache/Redis infrastructure	High
[71]	LLMCache (layer-wise)	Speedup / accuracy	3.1x speedup, under 0.5% accuracy loss	no layer-wise caching	Moderate

Each paper also has a quality tier based on the evaluation criteria in Section 2.4. A result from a Low-tier paper is not necessarily wrong. However, it may have a weaker evaluation, come from a less established venue, or have other limitations. Therefore, Low-tier results should be considered more carefully when comparing them with results from High-tier papers. The one Low-tier result in this table, paper [58], reports the largest headline speedup of any paper here, a 99.6% latency reduction. That is exactly the kind of number this tier column exists to flag. The result comes from a source this review's own rubric scored lower on venue rigor and evaluation depth than the seventeen papers around it, so it should not be weighted the same as the High-tier results in the same table.

6.1 Aggregate Patterns in Reported Results

Two important points can be seen from the table, even though no single paper reports them directly.First, the reported speedups are mostly between 1.6× and 3.1× across different systems and domains. This can be seen in Reuse [60], DynSplit-KV [65], LLMCache [71], and Temporal Semantic Caching [68]. This suggests that 1.6×–3.1× may be a practical speedup range for current semantic caching

systems. The extra cost of embeddings and verification may limit how much more speed these systems can achieve.

Second, the papers that report both average performance and tail latency show a similar problem. StepCache [67] and the Freshness-Aware Caching paper [45] both show that the worst-case requests improve much less than the average cases. For example, StepCache improves mean latency by almost 3×, but its p95 latency improves very little. The Freshness-Aware Caching paper also reports that its p95 fetch latency is 3.5× higher than its median. This suggests that the extra work needed for reliability and verification can have a bigger effect on the slowest requests, which are often very important in real systems.

7 Discussion and Research Gaps

7.1 Central Finding of the Review

After reviewing all 87 identified records and the 66 papers that met the inclusion criteria, five common patterns were found (Section 4). Each pattern shows a different problem with semantic caching. However, when these patterns are considered together, they point to one main research gap.

Across all 66 papers, none measures the actual energy used by semantic caching and compares it with the energy used by the direct LLM inference that it replaces. The papers often report latency, monetary cost, hit rate, and accuracy, but they rarely measure energy. Section 6 lists 18 papers with directly comparable quantitative results, and none reports an energy-based result. Since this review is motivated by green computing, this is an important missing area in the current research.

This gap leads to three main questions:

Q1. Under what workload and similarity-threshold conditions can semantic caching actually save energy after including the energy used for embeddings, vector search, and verification?

Q2. Is there a minimum cache hit rate below which semantic caching uses more energy than simply running the LLM for every query?

Q3. Does a high cache hit rate always mean high energy savings, or can a system have a high hit rate but still save little energy?

These are not new research ideas created by this review. They come directly from the problems and results found in the reviewed papers. The next part of this section explains how the five patterns lead to these questions and discusses other areas that are still not well studied.

7.2 Basis of the Central Research Questions

Q1 follows directly from Pattern P4 (reliability fixes add unmeasured cost). StepCache [67] is the clearest single example in the corpus. As Pattern P4 and Cluster B already show, its tail latency barely moves even though its mean latency improves substantially. A mean-only evaluation would have hidden the true cost of the reliability fix. Three more examples confirm the same gap in cost terms rather than latency terms. IC-Cache's [55] relevance-based selection carries an overhead that is never fully measured. Cortex's [6] LLM-judge verification step adds a cost that is never quantified. GuardCache [57] leaves latency unimproved in specific risk categories. Each is a verification or defense mechanism whose own cost is not measured against the benefit it protects. None of these costs are denominated in energy anywhere in the corpus, which is precisely what Q1 asks for. Q2 follows from the same pattern combined with a second, independent observation. Several papers report that cache effectiveness is lowest exactly when a system is newest. InstCache's [33] cold-start weakness, already noted in Cluster A, is one example: effectiveness only improves as traffic volume grows. Real-Time Query Understanding [61] reports a 72-hour cold-start period before its cache reaches representative performance. Verification and embedding overhead (Q1) is roughly a fixed cost per query, and hit rate is lowest during exactly the period these two papers describe. Put those together, and early deployment looks like the most likely point where a caching system could cost more energy than it saves. No paper in the corpus reports a hit-rate curve alongside a matching energy-cost curve precise enough to locate where that crossover actually falls. Q3 is not an inference from indirect evidence the way Q1 and Q2 are. It is raised directly, in almost these exact terms, by a paper already in this corpus. SCALM [15] states plainly that "the commonly used cache hit ratio metric is not always effective in calculating cost savings." This is a direct challenge to an assumption the rest of the corpus's reporting largely takes for granted: every hit-rate figure in Section 6's meta-analysis table is implicitly reported as if a higher number is straightforwardly better. This review's own quality-tier flagging in Section 6 reinforces the same concern from a different angle. The single largest headline speedup reported anywhere in the meta-analysis table, a 99.6% latency reduction [58], comes from the one Low-tier source in that table, the source this review's own rubric scored lowest on venue rigor and evaluation depth. That does not mean the number is wrong. It means the corpus's most striking result and its least independently verifiable result are the same paper, which is exactly the kind of mismatch Q3 asks the field to check systematically rather than notice by accident. If SCALM's caution is correct, some share of the headline results in this corpus may not translate into the savings they appear to represent. No paper in this review, including SCALM itself, tests that claim directly against a measured energy outcome. Q1 through Q3 are also the point at which this review's Green Computing motivation connects to the corpus rather than sitting outside it. One paper in this review's citation set, cited as background rather than as a corpus member, measures real energy and carbon costs of LLM efficiency techniques in industry [16], but it does not study caching. Q1 through Q3 describe exactly the study that paper's methodology would need to be pointed at semantic caching specifically to answer.

7.3 Further Research Gaps

The other four patterns raise open questions of their own, narrower in scope than Q1 through Q3 but part of the same overall picture.

Pattern P1 (threshold reliability): This is the most heavily corroborated pattern in the corpus. It is supported mathematically [9], architecturally [22], adversarially [42], [32], [3], and across four independent domains in Cluster D (Section 5.4) [43], [59], [64], [68]. Yet every proposed fix narrows the problem rather than resolving it. It remains an open question whether threshold-based similarity matching can be incrementally improved to an acceptable reliability level, or whether Biton et al.'s [9] NP-hardness result means the

field needs a structurally different decision mechanism rather than a better-tuned version of the current one.

Pattern P2 (context-blindness): Every context-aware fix reviewed reports a residual limitation in the same category it claims to fix [4], [17], [54], [38], [13], [27], [19]. It is not established whether context-blindness is simply a hard engineering problem, one the field has not solved carefully enough yet. It may instead reproduce Pattern P1's reliability question one level higher: adding conversational context might just move the same unreliability somewhere new. No paper in the corpus tests that possibility directly.

Pattern P3 (fixes that don't generalise): Each paper in the corpus optimises one layer of the caching pipeline, without claiming to generalise to the others. This review did not find a systematic taxonomy of failure modes anywhere in the 66 papers. What exists instead is a sequence of individually-discovered limitations. Whether the field is converging toward a complete solution or enumerating an open-ended list of edge cases remains unclear from the corpus alone. **Pattern P5 (staleness):** Now supported by four independent sources across three domains [26], [45], [64], [43], each treats staleness as a limitation to note rather than a problem to solve, and each measures it differently. A staleness benchmark comparable across systems and domains is not attempted by any paper in this corpus.

7.4 Summary

Of the open challenges this review identifies, Q1 through Q3 (Section 7.1) are its central finding. They are the most concrete, the most directly tied to this review's own Green Computing motivation, and, unlike the pattern-level questions in Section 7.3, almost entirely unaddressed by any measurement already present in the corpus, rather than partially addressed and merely unresolved. The pattern-level questions in Section 7.3 describe what is conceptually unresolved in the field. Q1 through Q3 describe what has not yet been measured at all.

8 Conclusion

This review searched three databases, arXiv, and a citation index across three rounds. Screening applied explicit inclusion and exclusion criteria to 84 records. The result is a 66-paper corpus organised into five clusters: the query-response caching core, adaptive threshold and eviction design, security and reliability, agentic and domain-specific applications, and a structurally distinct KV-cache literature retained as contrast evidence. A quality-appraisal rubric scored every included paper on venue rigor, evaluation depth, reproducibility, and limitation transparency, so that lower-confidence sources remained visible in the synthesis rather than silently dropped or silently trusted. Five patterns recur across this corpus independently of any single paper or cluster. No fixed similarity threshold reliably separates a correct cache hit from an incorrect one. Caching decisions made from a single query in isolation stay blind to the conversation around them. Each proposed fix in this literature narrows one part of the problem and leaves the next part for the following paper. Making a cache more reliable routinely costs something that goes unmeasured in the same evaluation that reports the reliability gain. And what happens when a cached result goes stale, when the world changes after a response is stored, remains understudied relative to how often the underlying assumption is made. The central finding of this review sits above all five patterns rather than beside them. Eighteen papers in this corpus report results precise enough to compare side by side, and not one denominates its result in energy. A literature that talks continuously about efficiency has not, as a field, measured the one thing a green-computing audience means by that word. This review turns that gap into three concrete questions. Under what conditions does semantic caching produce a net energy saving, once its own overhead is counted? Is there a minimum viable hit rate below which caching costs more energy than direct inference? And can a high hit rate be trusted to imply high savings at all? These are not proposed as this group's own research plan. They are offered as what the literature itself, read as a whole, leaves open. This review has its own limits, worth stating plainly rather than leaving implicit. We conducted screening and quality appraisal without a second, independent reviewer. That is normal for a small, independent systematic review, but it is a real difference from dual-reviewer protocols. We reconstructed the search strategy from screening records rather than a preserved query log, a limitation already disclosed in Section 2 and repeated here because it bears on how confidently the 87-record count should be read. Four included papers carry a venue that could not be confirmed with full certainty from the available text. And any review of a fast-moving, preprint-heavy literature is out of date the moment it closes its search window. Papers published after this review's third round are not reflected here. Semantic caching has a real and growing evidence base behind its latency and cost claims. What this review adds is a single, corpus-wide observation that evidence base does not yet cover: whether any of it saves energy at all.

9 Generative AI Usage Statement

This project used Claude, a generative AI assistant developed by Anthropic, at multiple stages of the work. This section discloses that use in full, as required for this course's term project report.

Areas of Claude Usage

Drafting and revising text: Claude drafted and revised every section of this review - Introduction, Methodology, Taxonomy, Cross-Cutting Synthesis, the five cluster syntheses, Meta-Analysis, Discussion, Conclusion, and the Abstract. Each draft went through multiple rounds of author review, correction, and approval before acceptance.

Corpus organisation and verification: Claude helped build and maintain the master corpus spreadsheet. This included extracting author, year, venue from the source PDFs. It also included cross-checking claims in the corpus, core contributions, stated limitations, and reported results, against the original papers. Several errors caught during this process, including an incorrect paper citation in the Meta-Analysis table and an undersold finding in one included paper, were found and corrected as a direct result of this verification work.

Reference management: Claude generated the .bib file used for citation management, built directly from the verified corpus data, and confirmed it compiles correctly under standard BibTeX tooling.

Editing for clarity and consistency: Claude edited the prose throughout for sentence length, tone, and consistency. It also checked

the final assembled document for duplicated content across sections, incorrect cross-references, and citation coverage against the full corpus.

Presentation preparation: Claude drafted the accompanying presentation guide and speaker scripts used to prepare this project's course presentation.

Areas Not Covered by Claude

Claude did not design the review's research questions, conduct the original literature search, or make the inclusion and exclusion decisions that produced the corpus. Those decisions, along with the interpretation of findings and the final content of every section, were made and approved by the authors. All factual claims, numbers, and conclusions in this report were reviewed against the source literature by the author group before inclusion. The authors take full responsibility for the accuracy and integrity of the final submitted work.

References

[1] Anonymous. Toolcacheagent: Accelerating LLM agent through intelligent tool call caching, 2026. Under review as a conference paper at ICLR 2026 (anonymous, double-blind).

[2] Yuyang Tian, Desen Sun, Yi Ding, and Sihang Liu. Cache your prompt when it's green — carbon-aware caching for large language model serving. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 10(1), 2026. doi: 10.1145/3788087.

[3] XiangFan Wu, Lingyun Ying, Guoqiang Chen, Yacong Gu, and Haipeng Qu. Cache me, catch you: Cache related security threats in LLM serving frameworks. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2026. doi: 10.14722/ndss.2026.242812.

[4] Jianxin Yan, Wangze Ni, Lei Chen, Xuemin Lin, Peng Cheng, Zhan Qin, and Kui Ren. Contextcache: Context-aware semantic cache for multi-turn queries in large language models, 2025. Preprint.

[5] Heejin Kim, Jeongha Lee, and Hyokyung Bahn. Rethinking I/O caching for large language model inference on resource-constrained mobile platforms. *Mathematics*, 13(24), 2025.

[6] Chaoyi Ruan, Chao Bi, Kaiwen Zheng, Ziji Shi, Xinyi Wan, and Jialin Li. Cortex: Achieving low-latency, cost-efficient remote data access for LLM via semantic-aware knowledge caching. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*, 2026.

[7] Mohanad Afiffy, Mohamed Waleed Fakhr, and Fahima A. Maghraby. Enhancing adversarial resilience in semantic caching for secure retrieval augmented generation systems. *Scientific Reports*, 16, 2026. doi: 10.1038/s41598-026-36721-w.

[8] Baran Atalar, Xutong Liu, Jinhang Zuo, Siwei Wang, Wei Chen, and Carlee Joe-Wong. Continuous semantic caching for low-cost LLM serving, 2026. Preprint.

[9] Dvir David Biton and Roy Friedman. From exact hits to close enough: Semantic caching for LLM embeddings, 2026. Preprint.

[10] Asmit Kumar Singh, Haozhe Wang, Santosh Attaluri, Tak Chiam, and Weihua Zhu. Asynchronous verified semantic caching for tiered LLM architectures, 2026. Preprint.

[11] Chen Wang, Xunzhuo Liu, Yue Zhu, Alaa Youssef, Priya Nagpurkar, and Huamin Chen. Category-aware semantic caching for heterogeneous LLM workloads, 2025. Preprint.

[12] Xutong Liu, Baran Atalar, Xiangxiang Dai, Jinhang Zuo, Siwei Wang, John C.S. Lui, Wei Chen, and Carlee Joe-Wong. Semantic caching for low-cost LLM serving: From offline learning to online adaptation, 2026. Preprint.

[13] Waris Gill, Mohamed Elidrisi, Pallavi Kalapatapu, Ammar Ahmed, Ali Anwar, and Muhammad Ali Gulzar. Meancache: User-centric semantic caching for LLM web services. In *2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2025. doi: 10.1109/IPDPS64566.2025.00117.

[14] Ramaswami Mohandoss. Context-based semantic caching for LLM applications. In *2024 IEEE Conference on Artificial Intelligence (CAI)*, 2024. doi: 10.1109/CAI59869.2024.00075.

[15] Jiaxing Li, Chi Xu, Feng Wang, Isaac M von Riedemann, Cong Zhang, and Jiangchuan Liu. Scalm: Towards semantic caching for automated chat services with large language models. In *2024 IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS)*, 2024. doi: 10.1109/IWQoS61813.2024.10682957.

[16] Pelin Rabia Kuran, Rumbidzai Chitakunye, Vincenzo Stoico, Ilja Heitlager, and Justus Bogner. Green LLM techniques in action: How effective are existing techniques for improving the energy efficiency of LLM-based applications in industry? In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE)*, 2026.

[17] Chengye Yu, Tianyu Wang, Zili Shao, and Song Jiang. Smartcache: Context-aware semantic cache for efficient multi-turn LLM inference. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025.

[18] Xiang Liu, Zhenheng Tang, Peijie Dong, Zeyu Li, Yue Liu, Bo Li, Xuming Hu, and Xiaowen Chu. Chunkkv: Semantic-preserving KV cache compression for efficient long-context LLM inference. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025.

[19] Qinghan Wang, Chuantao Li, Xiang Li, Chunxiao Wang, Jianglin Ma, Yang Zhang, Wenhui Yang, and Zhigang Zhao. Schemaaware-cache: A context- and schema-aware semantic caching framework for efficient text-to-SQL systems. In *2026 29th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*, 2026. doi: 10.1109/CSCWD68734.2026.11582648.

[20] Bang Fu and Di Feng. Gptcache: An open-source semantic cache for LLM applications. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*. Association for Computational Linguistics, 2023.

[21] Sajal Regmi and Chetan Phakami Pun. GPT semantic cache: Reducing LLM costs and latency via semantic embedding caching. In *Proceedings of the International Conference on Machine Learning (ICML), Workshop Track*, 2024. Workshop paper; exact workshop name unconfirmed from PDF.

[22] Luis Gaspar Schroeder, Aditya Desai, Alejandro Cuadron, Kyle Chu, Shu Liu, Mark Zhao, Stephan Krusche, Alfons Kemper, Matei Zaharia, and Joseph E. Gonzalez. vcache: Verified semantic prompt caching. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2026.

[23] Camille Couturier, Spyros Mastorakis, Haiying Shen, Saravan Rajmohan, and Victor Rühle. Semantic caching of contextual summaries for efficient question-answering with language models. In *Proceedings of the IEEE International Conference on Computer Communications and Networks (ICCCN)*, 2025. Preprint version at arXiv:2505.11271.

[24] Waris Gill, Justin Cechmanek, Tyler Hutcherson, Srijith Rajamohan, Jen Agarwal, Muhammad Ali Gulzar, Manvinder Singh, and Benoit Dion. Advancing semantic caching for LLMs with domain-specific embeddings and synthetic data, 2025. Preprint.

[25] Zafaryab Rasool, Scott Barnett, David Willie, Stefanus Kurniawan, Sherwin Balugo, Srikanth Thudumu, and Mohamed Abdelrazek. LLMs for test input generation for semantic applications. In *Proceedings of the 2024 IEEE/ACM 3rd International Conference on AI Engineering – Software Engineering for AI (CAIN 2024)*, 2024. doi: 10.1145/3644815.3644948.

[26] Siyuan Dang, Changfan Chen, Kejia Wu, Zhiyang Liu, and Yi Yang. Cachesense: Freshness-aware semantic caching with selective invalidation for LLM-serving backends, 2026. Venue not identified from available manuscript; treated as an unpublished/preprint manuscript.

[27] Nicholas Dominic and Bens Pardamean. Knowledge graph-enhanced semantic cache for low-latency and cost-effective inference in large language models. In *2024 International Conference on Information Management and Technology (ICIMTech)*, 2024. doi: 10.1109/ICIMTECH63123.2024.10780864.

[28] Guangda Liu, Chengwei Li, Jieru Zhao, Chenqi Zhang, and Minyi Guo. Clusterkv: Manipulating LLM KV cache in semantic space for recallable compression, 2025. Preprint.

[29] Shaul Dar, Michael J. Franklin, Björn T. Jónsson, Divesh Srivastava, and Michael Tan. Semantic data caching and replacement. In *Proceedings of the 22nd International Conference on Very Large Data Bases (VLDB)*, 1996.

[30] Boris Chidlovskii and Uwe M. Borghoff. Semantic caching of web queries. *The VLDB Journal*, 9:2–17, 2000.

[31] Qizheng Zhang, Michael Wornow, and Kunle Olukotun. Cost-efficient serving of LLM agents via test-time plan caching. In *ES-FoMo III: 3rd Workshop on Efficient Systems for Foundation Models, ICML 2025*, Vancouver, Canada, 2025.

[32] Guanlong Wu, Taojie Wang, Yao Zhang, Zheng Zhang, Jianyu Niu, Ye Wu, and Yinqian Zhang. When cache poisoning meets LLM systems: Semantic cache poisoning and its countermeasures. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2026. doi: 10.14722/ndss.2026.240200.

[33] Longwei Zou, Yan Liu, Jiamu Kang, Tingfeng Liu, Jiangang Kong, and Yangdong Deng. Instcache: A predictive cache for LLM serving, 2025. Preprint.

[34] Arun Iyengar, Ashish Kundu, Ramana Kompella, and Sai Nandan Mamidi. A generative caching system for large language models, 2025. Preprint.

[35] Yuxuan Zhu, Ali Falahati, David H. Yang, and Mohammad Mohammadi Amiri. Sentencekv: Efficient LLM inference via sentence-level semantic KV caching. In *Proceedings of the Conference on Language Modeling (COLM)*, 2025.

[36] Hafsa Akbar, Danish Athar, Muhammad Ayain Fida Rana, Chaudhary Hammad Javed, Zartash Afzal Uzmi, Ihsan Ayyub Qazi, and Zafar Ayyub Qazi. Semantic caching for improving web affordability, 2025. Preprint.

[37] Shervin Ghaffari, Zohre Bahranifard, and Mohammad Akbari. An ensemble embedding approach for improving semantic caching performance in LLM-based systems, 2025. Preprint.

[38] Jungwoo Kim, Minsang Kim, Jaeheon Lee, Chanwoo Moon, Heejin Kim, Taeho Hwang, Woosuk Chung, Yeseong Kim, and Sungjin Lee. Rethinking caching for LLM serving systems: Beyond traditional heuristics, 2025. Preprint.

[39] Tianyu Fu, Zihan Min, Hanling Zhang, Jichao Yan, Guohao Dai, Wanli Ouyang, and Yu Wang. Cache-to-cache: Direct semantic communication between large

language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2026.

[40] Yi Zhai, Dian Shen, Junzhou Luo, and Bin Yang. Toolcaching: Towards efficient caching for LLM tool-calling, 2026. Preprint.

[41] Varun Chillara, Dylan Kline, Christopher Alvares, Evan Wooten, Huan Yang, Shlok Khetan, Cade Bauer, Tré Guillory, Tanishka Shah, Yashodhara Dhariwal, Volodymyr Pavlov, and George Popstefanov. Semanticalli: Caching reasoning, not just responses, in agentic systems, 2026. Preprint.

[42] Zhixiang Zhang, Zesen Liu, Yuchong Xie, Quanfeng Huang, and Dongdong She. From similarity to vulnerability: Key collision attack on LLM semantic caching, 2026. Preprint.

[43] Laurent Bindschaedler. Semantic caching for OLAP via LLM-based query canonicalization (extended version). In *CEUR Workshop Proceedings*, 2026. Extended version.

[44] Shaoke Fang, Ziang Li, Wenfei Wu, Jiatong Ji, Qingsong Liu, and Ruizhi Pu. Not all tokens are worth caching: Learning semantic-aware eviction for LLM prefix caches, 2026. Preprint.

[45] Muhammad Mansoor, Tahir Ahmad, and Yeo-Chan Yoon. Risk-constrained freshness-aware semantic caching for open-web retrieval-augmented LLMs, 2026. Preprint.

[46] Henrique Lopes Nóbrega. Uso de cache semântico para economizar recursos em funcionalidades providas por LLMs. Bachelor's thesis (trabalho de conclusão de curso), ciência da computação, Universidade Federal de Campina Grande, Campina Grande, PB, Brazil, 2024.

[47] Abhijit Nayak and Raj Bhowmik. LLM reproducibility using three-way caching. In *2025 3rd International Conference on Foundation and Large Language Models (FLLM)*, 2025. doi: 10.1109/FLLM67465.2025.11390944.

[48] Qizheng Zhang, Michael Wornow, and Kunle Olukotun. Agentic plan caching: Test-time memory for fast and cost-efficient LLM agents. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025.

[49] Tao Kong, Hui Li, Yuxuan Zhao, Liping Li, Xiyue Gao, Qilong Wu, and Jiangtao Cui. Stscache: An efficient semantic caching scheme for time-series data workloads based on hybrid storage. *Proceedings of the VLDB Endowment (PVLDB)*, 2025.

[50] Tao Ren, Yiming Yao, Zheyuan Hu, and Jianwei Niu. Semcache: Semantic-aware cache sharing for efficient multi-user lora-adapted LLM inference at the edge. In *IEEE INFOCOM 2026 – IEEE Conference on Computer Communications*, 2026. doi: 10.1109/INFOCOM59046.2026.11571717.

[51] Nishanth Prakasan, Sudhanva Kalgundi, Ramya Bhat, and Mekhala Purohit. Collision-aware semantic hashing mechanisms for efficient caching of LLM outputs, 2026. TechRxiv preprint, posted 20 February 2026.

[52] Hao Yuan, Chentao Wu, Jie Li, and Minyi Guo. Turbo table: A semantic-aware cache acceleration system. In *2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)*, 2024. doi: 10.1109/ISPA63168.2024.00077.

[53] Jiacheng Liang, Yuhui Wang, Tanqiu Jiang, and Ting Wang. Lacache: Robust semantic caching for LLM serving, 2026. Preprint.

[54] Haoying Jin and Haoyang Feng. LLM-cache: An efficient context-aware semantic caching framework for distributed LLM inference services. In *2026 IEEE 46th International Conference on Distributed Computing Systems Workshops (ICDCSW)*, 2026. doi: 10.1109/ICDCSW72724.2026.00031.

[55] Yifan Yu, Yu Gan, Nikhil Sarda, Lillian Tsai, Jiaming Shen, Yanqi Zhou, Arvind Krishnamurthy, Fan Lai, Henry M. Levy, and David E. Culler. Ic-cache: Efficient large language model serving via in-context caching. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25)*, 2025.

[56] Denis S. Muradyan, Oleg A. Shutko, and Fedor V. Bushemelev. Adaptive web resource parsing system based on large language models using semantic caching. In *2026 XXIX International Conference on Soft Computing and Measurements (SCM)*, 2026. doi: 10.1109/SCM71573.2026.11625872.

[57] Shijing Hu, Xinyu Wang, Hengqi Guo, and Zhihui Lu. Guardcache: Memory-augmented adaptive input security for LLM semantic caching in FinTech. In *2025 IEEE Cyber Science and Technology Congress (CyberSciTech)*, 2025. doi: 10.1109/CyberSciTech68397.2025.00025.

[58] Agry Alfiah. LLM latency optimization on non-standard queries using hybrid semantic cache middleware. *International Journal of Social and Education (INJOSEDU)*, 3(4):483–488, 2026. ISSN 3047-6151.

[59] Jianfeng Zhang, Lingxi Cui, Yongqin Xu, Tong Wang, Yi Guo, Zhouzhi Yang, Huan Li, Ke Chen, and Lidan Shou. Chronosbi: Supercharging LLM-powered business intelligence pipelines with semantic caching and cost planning. In *Companion of the 2026 International Conference on Management of Data (SIGMOD Companion '26)*, Bengaluru, India, 2026.

[60] Zhuanglin Zheng, Yuxiang Zeng, Yunzhen Chi, and Yongxin Tong. Pick up where you left off: An efficient solution for continuous vector similarity search. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '26)*, Beijing / KDD '26, 2026.

[61] Praveen Kumar Alam. Semantic caching for mitigating LLM inference latency in real-time query understanding. *European Modern Studies Journal*, 9(5), 2025. ISSN 2522-9400. doi: 10.59573/emsj.9(5).2025.12.

[62] Henrique Lopes Nóbrega, David Candeia Medeiros Maia, and João Brunet. Evaluating semantic caching in practice: A study on a LLM-driven distributed application in a brazilian EdTech, 2025. Venue not confirmed from available manuscript (Brazilian CS venue, exact name unconfirmed).

[63] Md. Tareq Mahmood, Venkatesh Emani, Hangdong Zhao, and Shivaram Venkataraman. Demo of semweave: Semantic common expressions for LLM-powered query processing. In *Companion of the 2026 International Conference on Management of Data (SIGMOD Companion '26)*, Bengaluru, India, 2026.

[64] Ruiqi Zheng, Jinli Cao, Jiao Yin, and Hongzhi Yin. Scalrec: Semantic calibration for LLM-enabled cloud–device sequential recommendation, 2026. Preprint.

[65] Jiancai Ye, Jun Liu, Qingchen Li, Tianlang Zhao, Hanbin Zhang, Jiayi Pan, Ningyi Xu, and Guohao Dai. Dynsplit-KV: Dynamic semantic splitting for kvcache compression in efficient long-context LLM inference, 2026. Preprint.

[66] Abhishek Vijaya Kumar, Bhaskar Kataria, Byungsoo Oh, Emaad Manzoor, and Rachee Singh. Tvcache: A stateful tool-value cache for post-training LLM agents, 2026. Preprint.

[67] Azam Nouri. Stepcache: Step-level reuse with lightweight verification and selective patching for LLM serving, 2026. Preprint.

[68] Alimurtaza Merchant, Krish Veera, Sajal Kumar Goyla, Shambhawi Bhure, Dhaval Patel, and Kaoutar El Maghraoui. Evaluating temporal semantic caching and workflow optimization in agentic plan-execute pipelines, 2026. Preprint.

[69] Ali Noshad, Zishan Zheng, and Yinjun Wu. Mvr-cache: Optimizing semantic caching via multi-vector retrieval and learned prompt segmentation, 2026. Preprint.

[70] Aditeya Baral, Radoslav Ralev, Iliya Sotirov Zhechev, Srijith Rajamohan, and Jen Agarwal. Closing the calibration gap in semantic caching, 2026. Preprint.

[71] Harsh Vardhan Bansal. Llmcache: Layer-wise caching strategies for accelerated reuse in transformer inference. In *2025 13th International Conference on Intelligent Systems and Embedded Design (ISED)*, 2025. doi: 10.1109/ISED67359.2025.11405274.

[72] Heekyum Kim and Yuchul Jung. Entropy-guided KV caching for efficient LLM inference. *Mathematics*, 13(15), 2025. doi: 10.3390/math13152366.

[73] Can Wang, Dianbo Sui, Bolin Zhang, Xiaoyu Liu, Jiabao Kang, Zhidong Qiao, and Zhiying Tu. A framework for effective invocation methods of various LLM services. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING 2025)*, 2025.

[74] Zhiyuan Shi, Qibo Qiu, Feng Xue, Zhonglin Jiang, Li Yu, Jian Jiang, Xiaofei He, and Wenxiao Wang. Heterocache: A dynamic retrieval approach to heterogeneous KV cache compression for long-context LLM inference. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL 2026)*, 2026.

[75] Yuchu Chen, Yingchi Mao, Si Chen, Rongzhi Qi, Tianfu Pang, and Zhenxiang Pan. A cloud-edge collaborative system for efficient LLM fine-tuning with backbone activation caching. In *IEEE INFOCOM 2026 Workshops: Cloud-Network Convergence (CNC)*, 2026. doi: 10.1109/INFOCOM59046.2026.11571548.

[76] Minrui Xu, Dusit Niyato, Hongliang Zhang, Jiawen Kang, Zehui Xiong, Shiwen Mao, and Zhu Han. Cached model-as-a-resource: Provisioning large language model agents for edge intelligence in space–air–ground integrated networks. *IEEE Transactions on Networking*, 34, 2026. doi: 10.1109/TON.2025.3649068.

[77] Ning Yang, Wentao Wang, Lingtao Ouyang, and Haijun Zhang. Cooperative edge caching with large language model in wireless networks. *IEEE Transactions on Mobile Computing*, 2026. doi: 10.1109/TMC.2026.3687739.

[78] Harikrishnan C, Giridhar Sreenivasa Murthy, and Kumar Rangarajan. Augmenting keyword-based search in mobile applications using LLMs. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining (WSDM '24)*, 2024.

[79] Haoyu Wang, Wenxuan Ma, Bing Hu, Haozhe Li, Qingqian Si, Hongke Guo, Boyang Huang, Zhaoliang Zhu, You Zhang, Yu Zhou, and Xudong Zheng. Duallane: Fast and reliable LLM agents for interactive AIOps via dual-path planning. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '26)*, 2026.

[80] Aditya Gautam and Somya Bhargava. Multi-agent system for adversarial robustness and originality attribution in short-form videos. In *Proceedings of the 19th ACM International Conference on Web Search and Data Mining (WSDM '26)*, Boise, ID, USA, 2026.

[81] Yihan Dai, Dimitrios Stamatios Bouras, Haoxiang Jia, and Sergey Mechtaev. Statistical independence aware caching for LLM workflows. In *Companion Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE Companion)*, Rio de Janeiro, Brazil, 2026. doi: 10.1145/3786181.3788729.

[82] Haoxiang Zhang, Shi Chang, Arthur Leung, Kishanthan Thangarajah, Boyuan Chen, Hanan Lutfiyya, and Ahmed E. Hassan. Software performance engineering for foundation model-powered software. *ACM Transactions on Software Engineering and Methodology*, 2026.

[83] Filipe R. Cogo, Gustavo A. Oliva, and Ahmed E. Hassan. Compiler.next: A search-based compiler to power the AI-native future of software engineering. *ACM Transactions on Software Engineering and Methodology*, 2026.

[84] Hiroki Matsutani, Naoki Matsuda, and Naoto Sugiura. Accelerating local LLMs on resource-constrained edge devices via distributed prompt caching. In *Sixth European Workshop on Machine Learning and Systems (EuroMLSys '26)*, 2026.

[85] Wuyang Zhang, Yebo Cao, Yinzhi Jin, Zhen Luo, Jianming Ma, and Wangming Yuan. RAG-governor: Unified governance for trustworthy and cost-efficient retrieval-augmented generation with SLO guarantees. In *The 2nd International Conference on Digital Management and Information Technology (DMIT 2026)*, Beijing, China, 2026.

[86] Jinwook Yang, Junghyun Lee, Yeonsun Hong, and Hyojin Sung. Rapo: Retrieval-augmented phase ordering. In *Proceedings of the 27th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES '26)*, Boulder, CO, USA, 2026. doi: 10.1145/3814943.3816172.

[87] Ashmi Banerjee, Adithi Satish, Fitri Nur Aisyah, Wolfgang Wörndl, and Yashar Deldjoo. Collab-rec: An LLM-based agentic framework for balancing recommendations in tourism. *ACM Transactions on Recommender Systems*, 2026.